Universidad La Salle Bajío

Facultad de Ingenierías y Tecnologías

Investigación sobre KNN

Materia

Tópicos del big data

Nombre del estudiante

César Alberto Cázares Chaván

13 de octubre de 2026

1. Introducción y delimitación del tema

El aprendizaje automático permite utilizar datos para predecir resultados y reconocer relaciones entre observaciones. Dentro de este campo, KNN significa *k-nearest neighbors*, o k vecinos más cercanos. Su idea consiste en comparar un caso nuevo con ejemplos conocidos y utilizar los más próximos para decidir su respuesta. Esta investigación explica sus fundamentos, su funcionamiento y los criterios necesarios para aplicarlo de manera adecuada.

Aunque la consigna lo denomina modelo de agrupamiento, es necesario precisar su categoría. KNN, cuando predice una clase o un valor a partir de ejemplos etiquetados, pertenece al aprendizaje supervisado. El agrupamiento o *clustering* busca descubrir conjuntos sin una variable objetivo conocida. La búsqueda de vecinos también puede realizarse sin etiquetas y servir para construir relaciones de proximidad, pero esa búsqueda por sí sola no constituye un procedimiento completo de agrupamiento (scikit-learn developers, s. f.-a, s. f.-b).

Criterio	KNN	K-means
Objetivo	Predecir una clase o un valor.	Formar grupos de observaciones.
Datos	Ejemplos con respuesta conocida.	Variables sin etiquetas objetivo.
Significado de k	Cantidad de vecinos consultados.	Cantidad de grupos buscados.
Resultado	Predicción para un caso nuevo.	Asignación a un grupo y centroides.

Por ejemplo, asignar una compra a una categoría previamente definida a partir de compras etiquetadas es clasificación. Descubrir segmentos de clientes por sus hábitos de consumo es agrupamiento. Ambos problemas pueden depender de distancias, pero utilizan información diferente y producen resultados distintos. Confundirlos dificulta elegir las métricas y justificar la solución.

El objetivo general es comprender cómo KNN transforma la proximidad en una predicción. Los objetivos específicos son describir su origen, explicar el papel de k y las distancias, resolver un ejemplo e identificar sus ventajas y limitaciones. Se consultaron publicaciones académicas, documentación oficial y un laboratorio de los autores de un libro de aprendizaje estadístico. Las referencias finales permiten verificar las fuentes; el ejemplo numérico es de elaboración propia.

2. Antecedentes y funcionamiento de KNN

Cover y Hart publicaron en 1967 un análisis clásico de la clasificación por el vecino más cercano. Su artículo reconoce antecedentes de Fix y Hodges y estudia la regla que asigna al caso nuevo la categoría de la observación más próxima. Bajo los supuestos del trabajo y en el límite de muestras grandes, el error de esa regla queda acotado por el doble del error de Bayes. Se trata de un resultado teórico sobre 1-NN, no de una garantía de precisión para cualquier conjunto finito o configuración de KNN (Cover y Hart, 1967).

El error de Bayes representa el mínimo error de clasificación alcanzable si se conociera la distribución probabilística que genera los datos. La importancia del resultado histórico es mostrar que una regla local sencilla puede tener propiedades estadísticas relevantes. No implica que KNN siempre supere otros modelos ni que aumentar k produzca automáticamente mejores predicciones.

En una aplicación, cada registro se representa mediante un vector de características: medidas, cantidades o atributos transformados a una representación comparable. Además, los registros de entrenamiento tienen una respuesta conocida. El parámetro k indica cuántos ejemplos cercanos se consultan. Debe ser un entero positivo y no superar el número de observaciones disponibles en el entrenamiento de cada ajuste.

La secuencia de predicción es: **1)** representar y transformar el caso nuevo de la misma manera que el entrenamiento; **2)** calcular su distancia a los ejemplos; **3)** identificar los k más próximos; **4)** combinar sus respuestas; y **5)** devolver una predicción. En clasificación se utiliza la clase con más votos; en regresión, una media de los valores. La votación puede ser uniforme o dar mayor peso a los vecinos cercanos (scikit-learn developers, s. f.-a).

KNN suele describirse como aprendizaje basado en instancias: conserva ejemplos en lugar de resumirlos en una ecuación con pocos coeficientes. Es no paramétrico porque no presupone una forma funcional fija para la relación entre entradas y respuesta. Aun así, depende de decisiones como la representación, la distancia y k. La simplicidad de su regla no elimina la necesidad de validar esas decisiones.

En el laboratorio de clasificación de James et al. (2023) se comparan configuraciones con distintos valores de k y se evalúan sus predicciones. Esto permite observar que el desempeño depende del conjunto analizado. La elección debe apoyarse en datos reservados para evaluación, no solamente en la claridad intuitiva del algoritmo.

3. Distancias y preparación de los datos

La cercanía necesita una definición matemática. Para dos vectores x y z con m características, la distancia euclidiana puede escribirse como $d(x,z) = \sqrt{[\Sigma(x_j - z_j)^2]}$, donde la suma recorre j = 1, ..., m. La distancia Manhattan es $\Sigma|x_j - z_j|$. Ambas pertenecen a la familia Minkowski: $[\Sigma|x_j - z_j|^p]^{1/p}$, para p ≥ 1. Con p = 2 se obtiene la primera y con p = 1 la segunda (scikit-learn developers, s. f.-h).

Como ejemplo propio, entre (1, 2) y (4, 6) las diferencias son 3 y 4. La distancia euclidiana es $\sqrt{(9 + 16)} = 5$; Manhattan resulta 3 + 4 = 7. Ninguna cifra representa por sí misma una probabilidad. Son medidas de separación según una geometría. Cambiar la distancia puede modificar el orden de los vecinos y, por tanto, la respuesta final.

La escala de las variables resulta decisiva. Una diferencia de miles de unidades en una característica puede dominar diferencias pequeñas en otra, aunque ambas sean relevantes. El ejemplo oficial sobre escalamiento muestra que las fronteras de un clasificador de vecinos cambian al transformar variables con magnitudes distintas. Por ello, antes de interpretar la proximidad es necesario revisar unidades y rangos (scikit-learn developers, s. f.-c).

La estandarización convierte una variable mediante $z = (x - \mu) / \sigma$, con media μ y desviación estándar σ estimadas en entrenamiento. El reescalamiento mínimo-máximo utiliza (x - mínimo) / (máximo - mínimo). Las variables constantes requieren tratamiento especial para evitar dividir entre cero. Estos métodos ajustan las magnitudes, pero no hacen que una variable irrelevante se vuelva útil (scikit-learn developers, s. f.-d).

También se deben revisar valores faltantes, unidades inconsistentes, duplicados y observaciones atípicas. Una categoría nominal, como una ciudad, no debería convertirse arbitrariamente en números consecutivos: esa representación inventaría diferencias y un orden. La codificación debe conservar el significado de los atributos y ser compatible con la distancia utilizada. Las decisiones de limpieza deben justificarse según el contexto.

Para evitar fuga de información, primero se separan los datos y después se ajustan las transformaciones con entrenamiento. Las mismas reglas se aplican a los datos nuevos. Durante validación cruzada, el escalador se vuelve a ajustar dentro de cada partición de entrenamiento. Una *Pipeline* ayuda a mantener este orden y evita que información de evaluación influya en la preparación (scikit-learn developers, s. f.-e).

4. Ejemplo numérico de clasificación y regresión

El siguiente ejemplo es sintético y de elaboración propia. Se dispone de dos características numéricas, x_1 y x_2, en una escala comparable, y de dos clases conocidas, A y B. Se desea clasificar el punto nuevo Q = (3, 3). Se emplean distancia euclidiana y votación uniforme. La tabla presenta los registros ordenados por proximidad a Q.

Registro	Coordenadas	Clase	Distancia a Q
P1	(3, 2)	B	1.000
P2	(2, 2)	A	$\sqrt{2} \approx 1.414$
P3	(2, 4)	A	$\sqrt{2} \approx 1.414$
P4	(5, 3)	B	2.000
P5	(6, 4)	B	$\sqrt{10} \approx 3.162$

Para P2, el cálculo es $d(Q,P2) = \sqrt{[(3 - 2)^2 + (3 - 2)^2]} = \sqrt{2}$. Con k = 3 se eligen P1, P2 y P3. La clase A recibe dos votos y B recibe uno; por ello, la predicción es A. Los puntos P2 y P3 empatan en distancia, pero ambos entran en este vecindario y su orden no modifica la respuesta.

Con k = 1 solo se consulta P1 y la predicción cambia a B. Con k = 5 se consideran todos los registros: B obtiene tres votos y A dos, por lo que el resultado también es B. Este ejemplo demuestra que k altera la región utilizada para decidir. No demuestra cuál configuración sería más precisa: para determinarlo se necesitan etiquetas reales de casos nuevos y una evaluación separada.

Si se usan pesos inversos a la distancia con k = 3, B recibe un peso de 1/1 = 1 y A recibe $1/\sqrt{2} + 1/\sqrt{2} \approx 1.414$. A sigue ganando. En otra distribución, un solo punto muy cercano podría cambiar el resultado ponderado. Los casos con distancia cero requieren una regla específica para evitar la división entre cero.

En regresión se conservan los mismos tres vecinos, pero sus respuestas son valores numéricos. Si P1, P2 y P3 tienen respuestas 10, 12 y 14, la predicción uniforme es (10 + 12 + 14) / 3 = 12. Una versión ponderada calcula $\Sigma(w_i y_i) / \Sigma w_i$. La combinación de valores cambia, mientras que la búsqueda de proximidad sigue siendo la misma.

La proporción de votos puede servir como estimación local de pertenencia a una clase. En este caso, 2/3 de los vecinos corresponden a A. Esa fracción no acredita que exista una probabilidad real y calibrada de 66.7 %; su utilidad depende del muestreo, la representación y la validación del modelo.

5. Selección de k y evaluación del desempeño

Un k pequeño produce decisiones muy locales y puede reaccionar a ruido o etiquetas erróneas. Un k grande suaviza la respuesta, pero puede mezclar regiones distintas y perder patrones locales. La elección representa un equilibrio entre flexibilidad y estabilidad. Usar un valor impar reduce los empates de votos en clasificación binaria con pesos uniformes; no impide todos los empates en problemas con más clases ni resuelve empates en distancias.

Una estrategia consiste en reservar un conjunto de prueba y seleccionar los hiperparámetros dentro del entrenamiento mediante validación cruzada. En cada ronda se ajusta el procedimiento con una parte de los datos y se evalúa con otra. Para clasificación, las particiones estratificadas conservan aproximadamente la proporción de clases. Si hay registros de una misma persona o datos temporales, la partición debe respetar esos grupos o el orden cronológico (scikit-learn developers, s. f.-f).

Se pueden comparar, por ejemplo, k = 1, 3, 5, 7 y 9, distancias Manhattan y euclidiana, y pesos uniformes o por distancia. Esos valores son candidatos ilustrativos, no una receta universal. Una búsqueda en cuadrícula evalúa combinaciones según una métrica previamente elegida. El escalamiento debe estar dentro del procedimiento evaluado. Tras seleccionar la configuración, se reajusta con el entrenamiento disponible y se realiza la evaluación final en prueba (scikit-learn developers, s. f.-g).

En clasificación, la exactitud o *accuracy* mide la proporción de aciertos. La precisión indica qué fracción de los positivos predichos era realmente positiva; la sensibilidad o *recall* señala qué fracción de los positivos reales fue detectada. F1 combina precisión y sensibilidad mediante su media armónica. La matriz de confusión identifica los tipos de error. En regresión, MAE resume errores absolutos y RMSE penaliza más los grandes, manteniendo las unidades del objetivo (scikit-learn developers, s. f.-i).

Un ejemplo propio muestra el problema del desbalance: si hay 95 casos de A y 5 de B, predecir siempre A produce 95 % de exactitud, pero sensibilidad cero para B. Ese resultado no sería satisfactorio cuando interesa reconocer B. La elección de métricas debe responder al propósito y al costo de los errores; una sola cifra puede ocultar fallas importantes.

Un informe reproducible debe indicar variables utilizadas, transformaciones, método de partición, candidatos comparados, criterio de selección y resultados finales. También debe examinar errores por clase y variabilidad entre particiones. Elegir repetidamente con el conjunto de prueba termina usándolo como validación y vuelve optimista la estimación de desempeño (scikit-learn developers, s. f.-e).

6. Aplicaciones, limitaciones y conclusiones

KNN puede estudiarse en problemas donde la similitud entre registros aporta información útil. El conjunto Iris ofrece un caso educativo: contiene 150 observaciones, cuatro características medidas en centímetros y tres clases de plantas. Permite practicar clasificación a partir de medidas de sépalos y pétalos. Es un recurso pequeño para aprender y no una prueba de que el método tendrá el mismo desempeño en otros contextos (Fisher, 1936, conjunto documentado por UCI).

En un escenario hipotético de comercio, KNN podría asignar una categoría conocida a un artículo a partir de productos etiquetados y atributos comparables. En otro de regresión, podría estimar una cantidad con ejemplos similares. Estas propuestas requieren datos adecuados y evaluación: que dos registros se parezcan en las variables elegidas no garantiza que tengan la misma respuesta ni establece causalidad.

Sus ventajas prácticas incluyen una regla de decisión sencilla de explicar mediante vecinos concretos y la posibilidad de representar relaciones locales. Sus dificultades incluyen almacenar ejemplos, consultar distancias y mantener una representación coherente. La biblioteca scikit-learn ofrece herramientas de clasificación, regresión y selección de modelos con una interfaz común; su publicación académica describe el proyecto y su propósito como herramienta de aprendizaje automático (Pedregosa et al., 2011).

Con n ejemplos y m características, calcular por fuerza bruta todas las distancias para una consulta supone aproximadamente O(nm) operaciones; seleccionar u ordenar vecinos añade trabajo. Guardar una matriz densa exige espacio del orden de O(nm). Estructuras como KDTree permiten organizar observaciones y efectuar búsquedas, aunque el beneficio depende de los datos y la dimensión. El parámetro de tamaño de hoja modifica la estructura, la memoria y el costo de consulta (scikit-learn developers, s. f.-h).

En muchas dimensiones, las nociones habituales de cercanía pueden perder utilidad y los datos necesarios para representar vecindarios pueden crecer considerablemente. Además, variables sin señal, ruido y clases poco representadas pueden deteriorar las decisiones. Por ello, en un proyecto de big data conviene medir memoria y tiempo de respuesta, revisar atributos y comparar alternativas. La facilidad de construir un clasificador no garantiza que sea eficiente a gran escala.

Conclusión. KNN predice mediante la proximidad a ejemplos conocidos. Comprenderlo exige distinguirlo de los métodos de agrupamiento, definir una distancia adecuada y justificar la elección de k. El ejemplo desarrollado muestra que una misma observación puede recibir respuestas diferentes al cambiar el vecindario. Su valor práctico depende de la calidad de los datos y de una evaluación sin fuga de información. Es una base útil para estudiar aprendizaje supervisado y un candidato razonable cuando las relaciones locales son informativas, siempre sujeto a comparación empírica con otras soluciones.

Referencias bibliográficas

Publicaciones académicas reales y verificables

Cover, T. M., y Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory, 13*(1), 21-27. https://doi.org/10.1109/TIT.1967.1053964
Texto consultado: PDF en el repositorio de Stanford.

James, G., Witten, D., Hastie, T., Tibshirani, R., y Taylor, J. (2023). *An introduction to statistical learning: With applications in Python.* Springer. https://doi.org/10.1007/978-3-031-38747-0
Material consultado: laboratorio de los autores, Logistic Regression, LDA, QDA, and KNN.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830. Artículo en JMLR.

Fisher, R. A. (1936). *Iris* [Conjunto de datos]. UCI Machine Learning Repository. https://doi.org/10.24432/C56C76. Ficha consultada.

Páginas de documentación oficial consultadas

Las siguientes entradas corresponden a páginas distintas consultadas el 13 de octubre de 2026. Se utiliza “s. f.” porque las páginas no presentan una fecha de publicación estable. Las letras permiten relacionarlas con las citas del desarrollo. Los enlaces son interactivos.

Referencias: páginas web consultadas

scikit-learn developers. (s. f.-a). *Nearest Neighbors.* Scikit-learn. Recuperado el 13 de octubre de 2026, de documentación oficial.

scikit-learn developers. (s. f.-b). *Clustering.* Scikit-learn. Recuperado el 13 de octubre de 2026, de documentación oficial.

scikit-learn developers. (s. f.-c). *Importance of Feature Scaling.* Scikit-learn. Recuperado el 13 de octubre de 2026, de documentación oficial.

scikit-learn developers. (s. f.-d). *Preprocessing data.* Scikit-learn. Recuperado el 13 de octubre de 2026, de documentación oficial.

scikit-learn developers. (s. f.-e). *Common pitfalls and recommended practices.* Scikit-learn. Recuperado el 13 de octubre de 2026, de documentación oficial.

scikit-learn developers. (s. f.-f). *Cross-validation: evaluating estimator performance.* Scikit-learn. Recuperado el 13 de octubre de 2026, de documentación oficial.

scikit-learn developers. (s. f.-g). *Tuning the hyper-parameters of an estimator.* Scikit-learn. Recuperado el 13 de octubre de 2026, de documentación oficial.

scikit-learn developers. (s. f.-h). *KDTree.* Scikit-learn. Recuperado el 13 de octubre de 2026, de documentación oficial.

scikit-learn developers. (s. f.-i). *Metrics and scoring: quantifying the quality of predictions.* Scikit-learn. Recuperado el 13 de octubre de 2026, de documentación oficial.